Iconic CAD Workflow Proof of Concept

Full Video

Claude Conversation

Google Drive with files

Collin DeSantis OSE 3-18-2026

Goals & Setup

Goals:

- Run a quick proof of concept test of the Iconic CAD workflow to get up to speed with Marcin
- Understand the premise of the compiler & Inkscape SVG layout
- Find any possible improvements

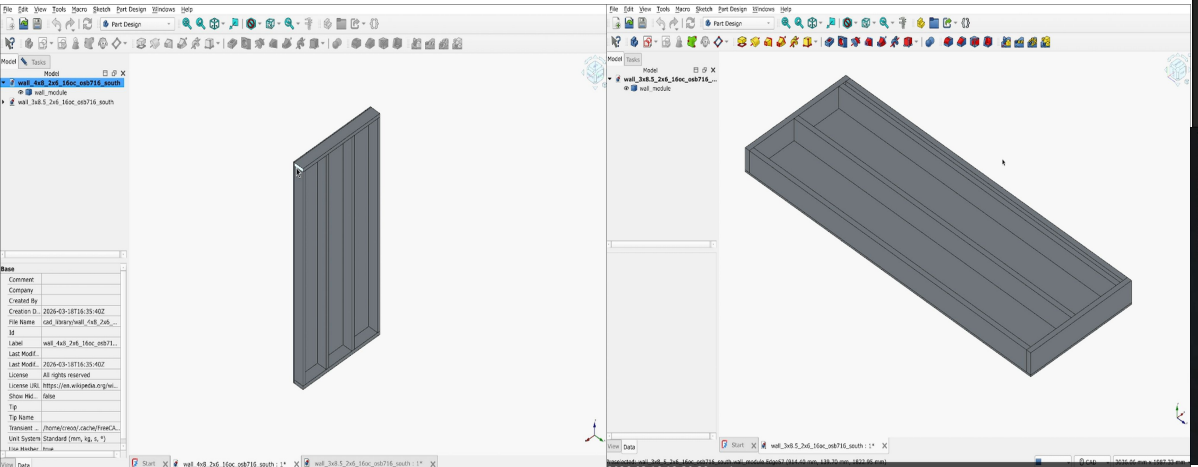
Setup:

1. Ensured install of freecad, python, python-yaml packages
2. Pulled files from wiki (Iconic_cad_icons.zip, project.zip)
3. Pulled files from AI w/ tar (generation and validation test files)

```
~
❯ sudo pacman -S freecad
[sudo] password for creoo:
```

Test 1: Part Library Generation & Validation Test

The goal is to generate your own cad_library folder in a contained test separate from the actual house generation test. The test went well, aside from the AI's script having a unit mismatch. I viewed the generated CAD files in FreeCAD to verify their correctness.



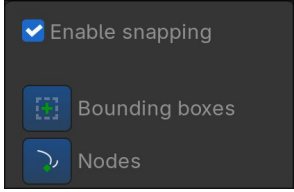
Diagnosed unit conversion mismatch and object naming discrepancies >

Your geometry is actually perfect — this is purely a unit mismatch bug in the validator. Look at the numbers:

- Expected 48.0 inches, got 1219.2mm → 1219.2 / 25.4 = **exactly 48 inches**
- Expected 96.0 inches, got 2438.4mm → 2438.4 / 25.4 = **exactly 96 inches**
- Expected 5.9375 inches, got 150.8125mm → 150.8125 / 25.4 = **exactly 5.9375 inches**

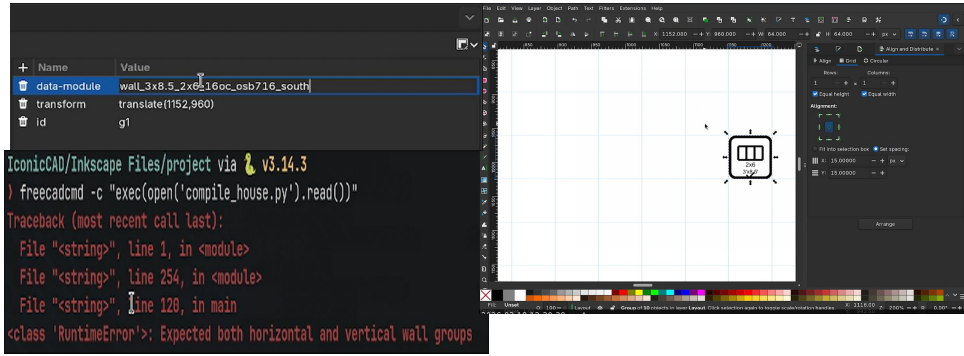
```
~/IconicCAD/iconic-cad-test via 🐍 v3.14.3
❯ freecadcmd -c "import sys; sys.argv=['validate_wall_geometry.py','wall_instances.yaml']; exec(open('validate_wall_geometry.py
').read())"
Importing project files......
Postprocessing.....                                    (100 %)
Importing project files......                          (100 %)
Postprocessing.....                                    (100 %)
Importing project files......                          (100 %)
Postprocessing.....                                    (100 %)
Geometry validation FAILED:
  wall_4x8_2x6_24oc_osb716_south: width mismatch expected 48.0 got 1219.1999999999998
  wall_4x8_2x6_24oc_osb716_south: height mismatch expected 96.0 got 2438.3999999999996
  wall_4x8_2x6_24oc_osb716_south: thickness mismatch expected 5.9375 got 150.8125
  wall_4x8_2x6_24oc_osb716_south: invalid stud count 0
  wall_4x8_2x6_24oc_osb716_south: expected 1 OSB panel, found 0
  wall_4x8_2x6_16oc_osb716_south: width mismatch expected 48.0 got 1219.1999999999998
  wall_4x8_2x6_16oc_osb716_south: height mismatch expected 96.0 got 2438.3999999999996
  wall_4x8_2x6_16oc_osb716_south: thickness mismatch expected 5.9375 got 150.8125
  wall_4x8_2x6_16oc_osb716_south: invalid stud count 0
  wall_4x8_2x6_16oc_osb716_south: expected 1 OSB panel, found 0
  wall_3x8.5_2x6_16oc_osb716_south: width mismatch expected 36.0 got 914.4
  wall_3x8.5_2x6_16oc_osb716_south: height mismatch expected 102.0 got 2590.7999999999997
  wall_3x8.5_2x6_16oc_osb716_south: thickness mismatch expected 5.9375 got 150.8125
  wall_3x8.5_2x6_16oc_osb716_south: invalid stud count 0
  wall_3x8.5_2x6_16oc_osb716_south: expected 1 OSB panel, found 0
                                                       (100 %)
```

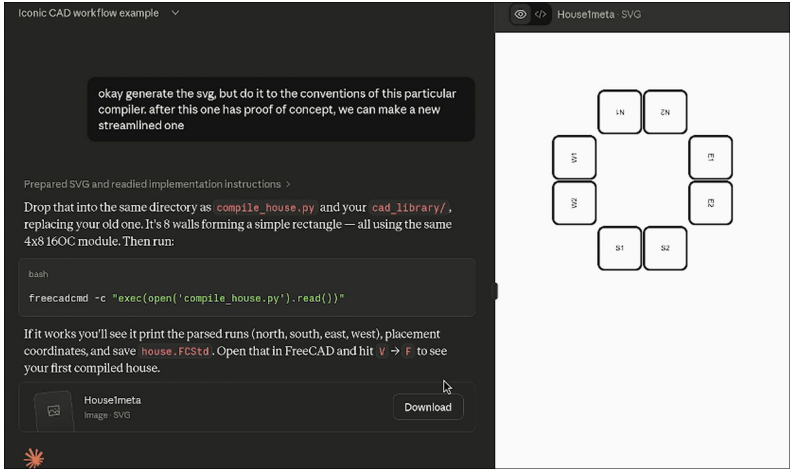
* Ensure snap by bounding box is on



Test 2, Step 1: SVG Layout in Inkscape

The goal is to layout wall modules in an SVG grid that the compiler can read and translate into 3D CAD files. This requires pasting in images that represent wall modules and adding metadata to them. I attempted to generate a CAD file with only one wall - then realized the compiler in its current state required walls facing all four directions - it scans the building in a loop. Thus discovering the ‘ports’ dilemma. Ports would be actual connection points on the CAD files so that this loop method could be bypassed. Think legos. I quickly realized this could be tedious - so I had claude generate me a simple proof of concept SVG image with metadata added in - this way I could test the loop method.

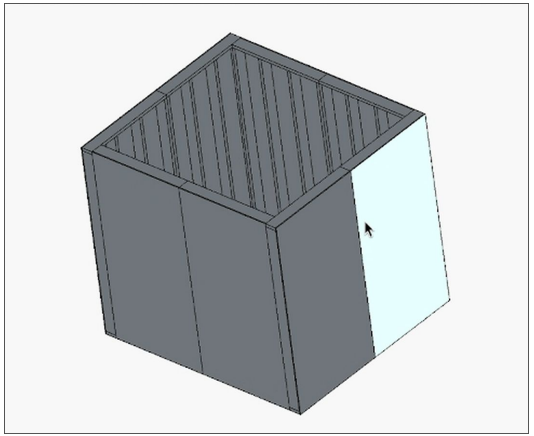


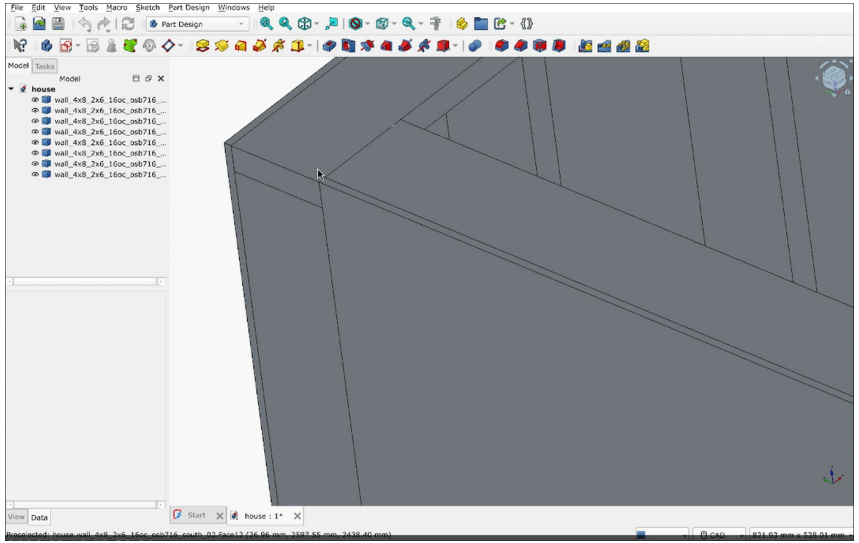


I believe that ports will be a superior method - and I will attempt to achieve this with claude code in a following test.

Test 2, Step 2: Generate CAD file with AI-made SVG

Goal: run the compiler on the AI-generated SVG file to achieve proof of concept of the loop-generation style. The compiler worked great here, and generated the small house.





Observations & Next Steps

Key Takeaways:

The core pipeline works. YAML generated CAD is solid and compiler assembled simple house easily.

Some pain points include manual XML input, rectangular only constraint. (Unless I am misunderstanding these parts of the process).

Possible Next Steps:

Auto generating SVGs from the yaml instances with metadata baked in?

Attempt ‘ports’ style connections between adjacent icons.

General streamlining with AI? Could claude code be useful here?